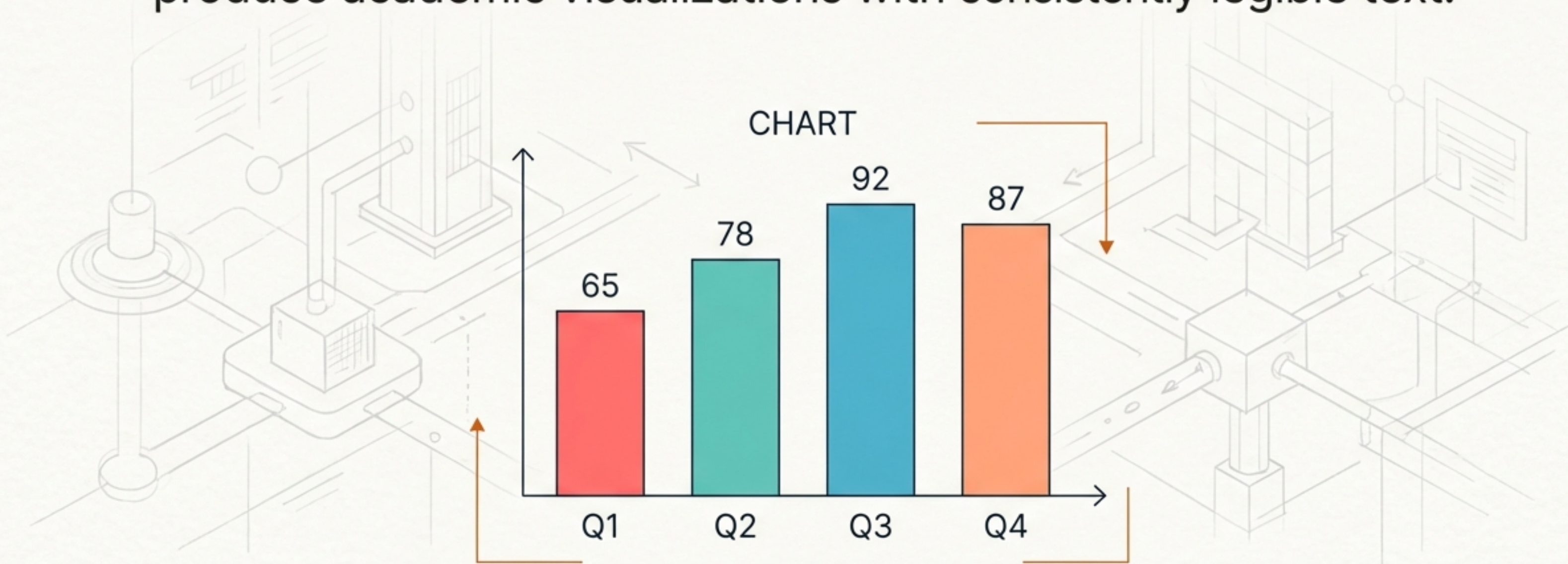


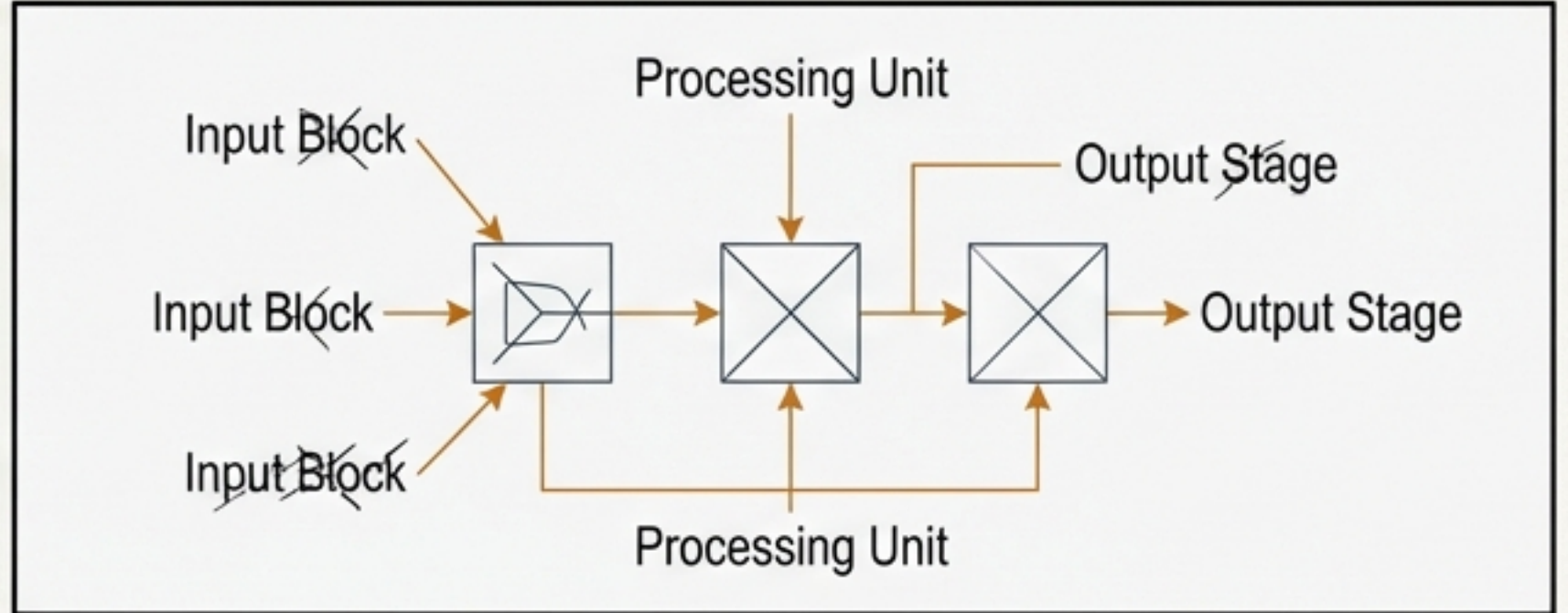
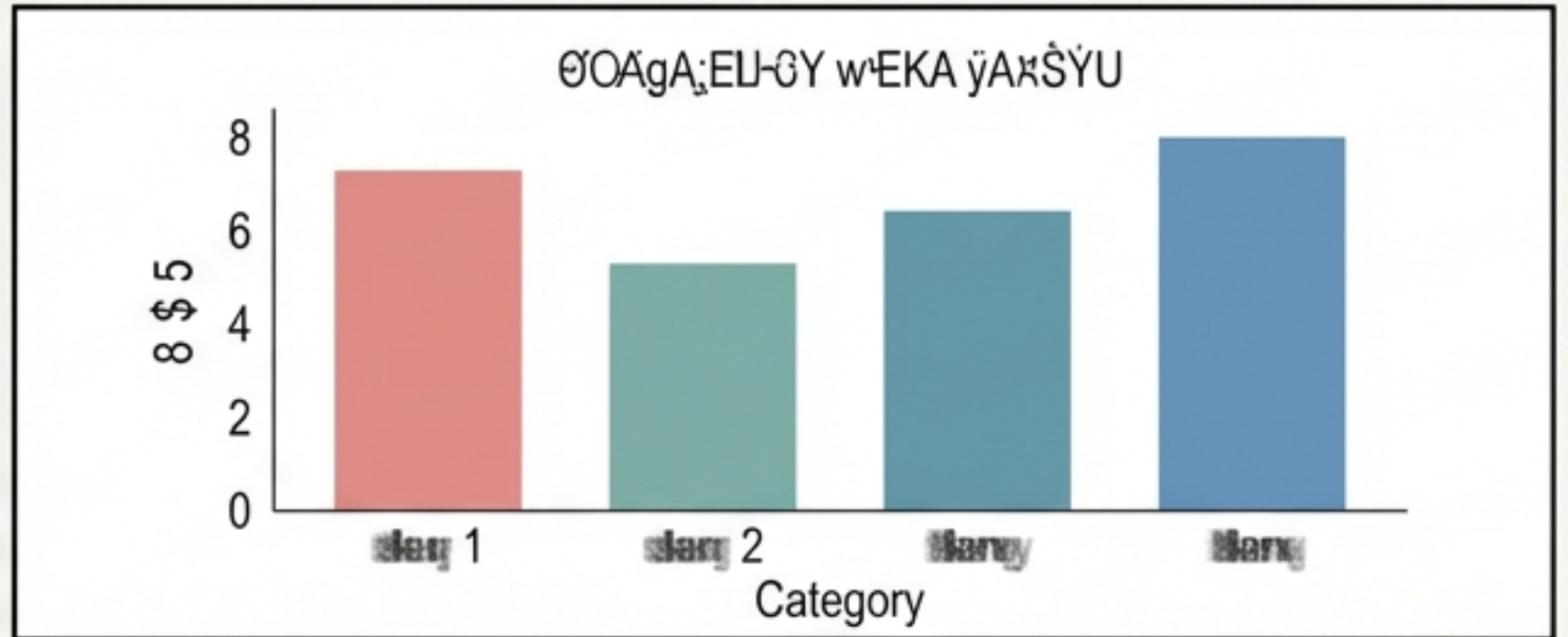
Solving Text in Diffusion Models: A Hybrid Generation & Rendering Approach

A technical walkthrough for fine-tuning Stable Diffusion to produce academic visualizations with consistently legible text.



Diffusion Models Natively Struggle with Coherent Text

Standard diffusion models often produce garbled, nonsensical, or misshapen text, making them unreliable for information-dense graphics like charts and diagrams.



The Solution: A Hybrid Approach that Decouples Visual Generation from Text Rendering

Instead of forcing the diffusion model to render text, we train it to generate text-free visuals. Clean, readable text is then programmatically overlaid in a deterministic post-processing step.



A Three-Phase Implementation

Phase 1: Synthetic Data Generation

We create a high-quality dataset of academic visualizations where text is programmatically added *after* the base graphic is rendered. The model never sees text rendered by a human or another AI.

Phase 2: Efficient Model Fine-Tuning

We fine-tune a Stable Diffusion model on this text-free dataset using LoRA, teaching it the visual structure of charts and diagrams.

Phase 3: Two-Step Inference

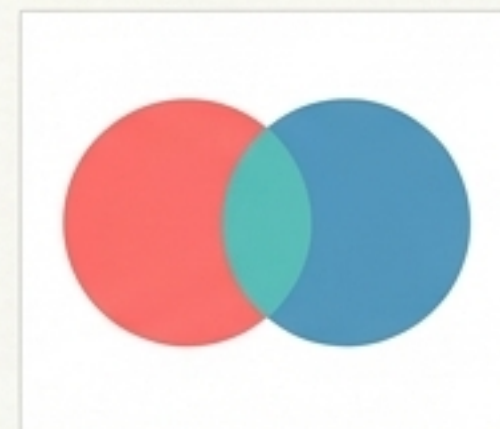
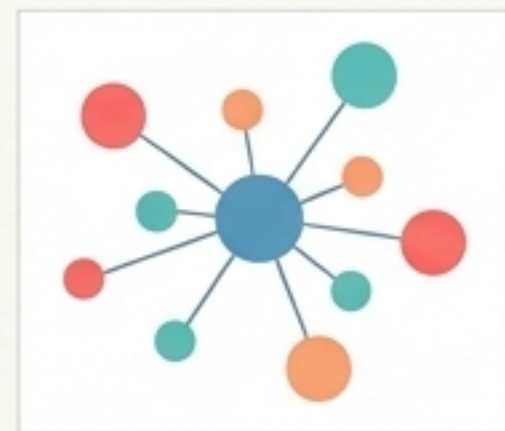
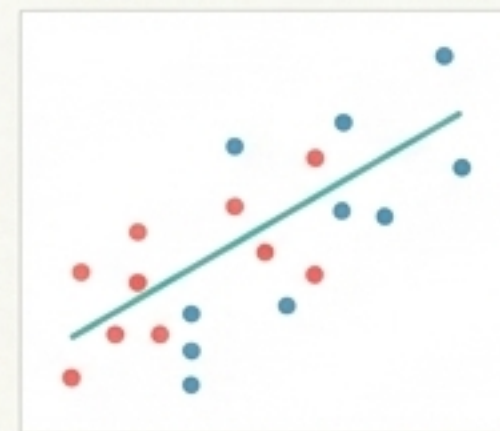
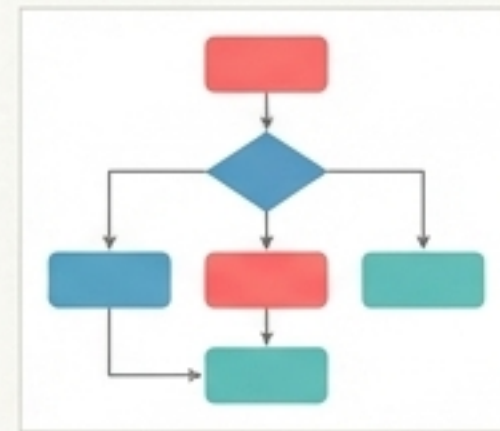
During inference, we first generate the text-free visual from a prompt, then use a helper utility to add the final text overlays.

Phase 1: Forging the Data with `AcademicVisualizationGenerator`

The foundation is a synthetic dataset generated using Matplotlib for clean, consistent base visuals. The text descriptions for training are generated alongside the images.

Key Features

- **7 Visualization Types:** Generates flowcharts, bar charts, line graphs, pie charts, scatter plots, network diagrams, and Venn diagrams.
- **Paired with Descriptions:** Each generated image is saved with a detailed text prompt, e.g., *'Bar chart with four bars labeled Q1, Q2, Q3, Q4 with values 65, 78, 92, 87.'*
- **Output:** The process results in an image folder and a `metadata.json` file, creating a structured dataset for training.



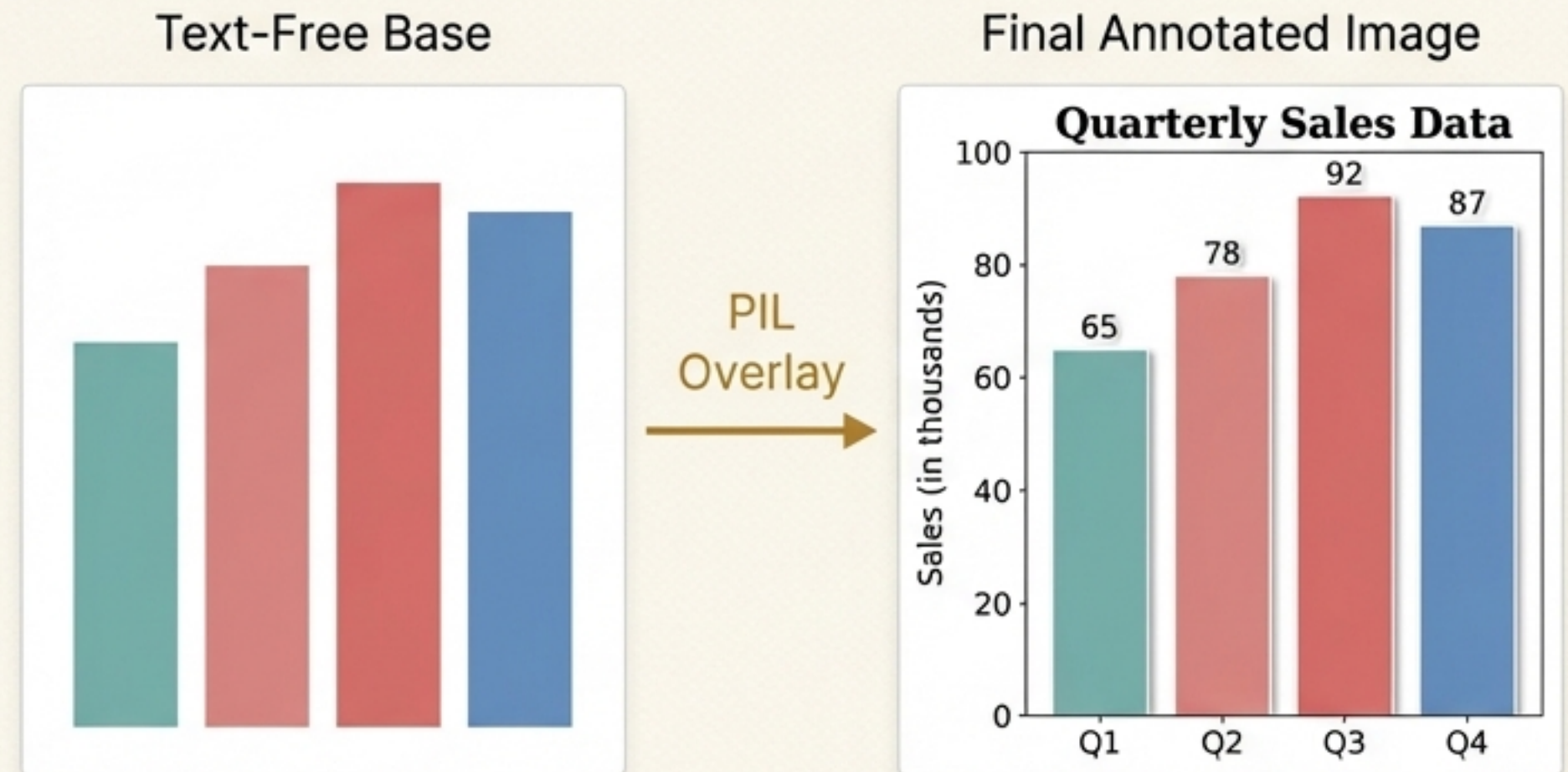
The `TextOverlayHelper`: Ensuring Perfect Readability with PIL

A dedicated Python class using the Pillow (PIL) library handles all text rendering. This ensures crisp, well-placed, and readable text on every generated image.

Core Functionality

- **Font Loading:** Loads system fonts with fallbacks for cross-platform consistency.
- **Readability First:** Implements a `draw\_text\_with\_shadow` function to guarantee text is legible against any background color.
- **Specialized Methods:** Includes specific functions for each chart type (`add\_flowchart\_labels`, `add\_bar\_chart\_labels`, etc.) to handle unique layout requirements.

```
def draw_text_with_shadow(
    self, draw, text, position, font,
    fill='black',
    shadow_color='white',
    shadow_offset=2
):
    # ... implementation ...
```

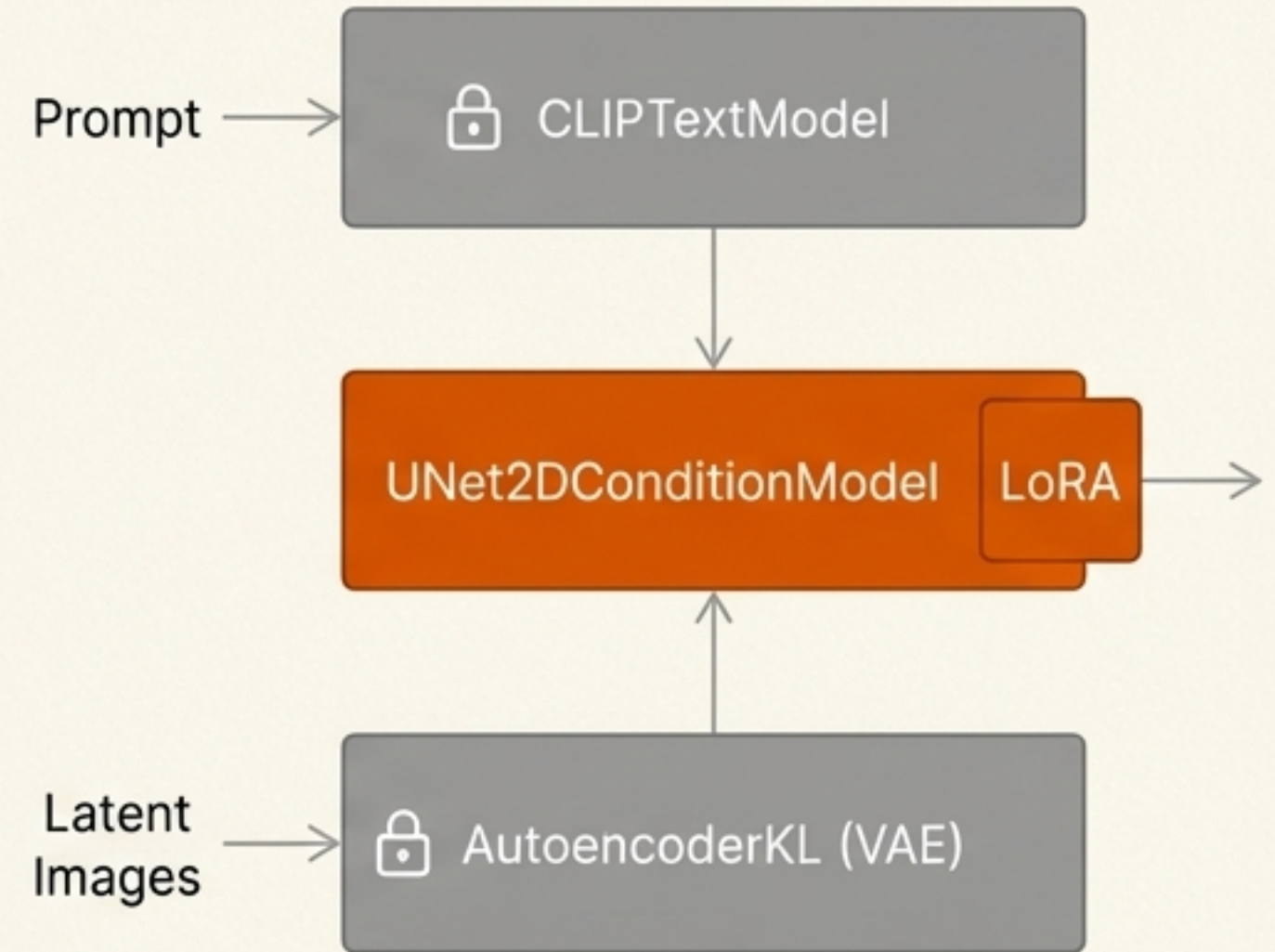


Phase 2: Setting Up the `AcademicVizModelTrainer`

We fine-tune a pre-trained Stable Diffusion model to specialize in generating academic visuals. The key is to train only the components necessary for learning visual styles, not language or text rendering.

Setup Details

- **Base Model:** `runwayml/stable-diffusion-v1-5`
- **Core Components:**
 - `CLIPTokenizer` & `CLIPTextModel` for processing input prompts.
 - `AutoencoderKL` (VAE) for encoding images into the latent space.
 - `UNet2DConditionModel` as the core image generator.
- **Frozen Weights:** The Text Encoder and VAE weights are frozen (`requires\_grad\_(False)`). Training is focused exclusively on the UNet.



Fine-Tuning with Surgical Precision via LoRA

To maximize efficiency, we adapt the UNet using Low-Rank Adaptation (LoRA). This avoids retraining the entire model by injecting small, trainable matrices into specific layers.

LoRA Configuration (`peft` library)

- **Rank** (`r`): 16
- **Alpha** (`lora_alpha`): 32
- **Target Modules:** `['to_k', 'to_q', 'to_v', 'to_out.0']`
- **Bias:** `'none'`

The Efficiency Payoff

Trainable Parameters:
3,188,736

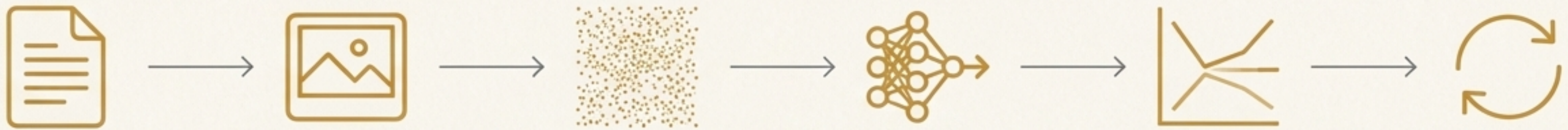
(out of 862,709,700 total UNet parameters)

This means we fine-tune less than **0.4%** of the model.

The Training Loop: Teaching the UNet to Denoise Visuals

The model is trained for 5 epochs. In each step, it learns to predict the noise that was added to a clean image's latent representation, guided by the text prompt.

Simplified Training Step:



1. Encode Prompt

The text prompt is converted into text embeddings via the frozen `CLIPTextModel`.

2. Encode Image

The training image is converted into latents by the frozen `VAE`.

3. Add Noise

Random noise is added to the latents based on a random timestep.

4. Predict Noise

The trainable LoRA-adapted `UNet` takes the noisy latents, timestep, and text embeddings to predict the original noise.

5. Calculate Loss

Mean Squared Error (`mse_loss`) is calculated between the `UNet`'s prediction and the actual noise.

6. Backpropagate

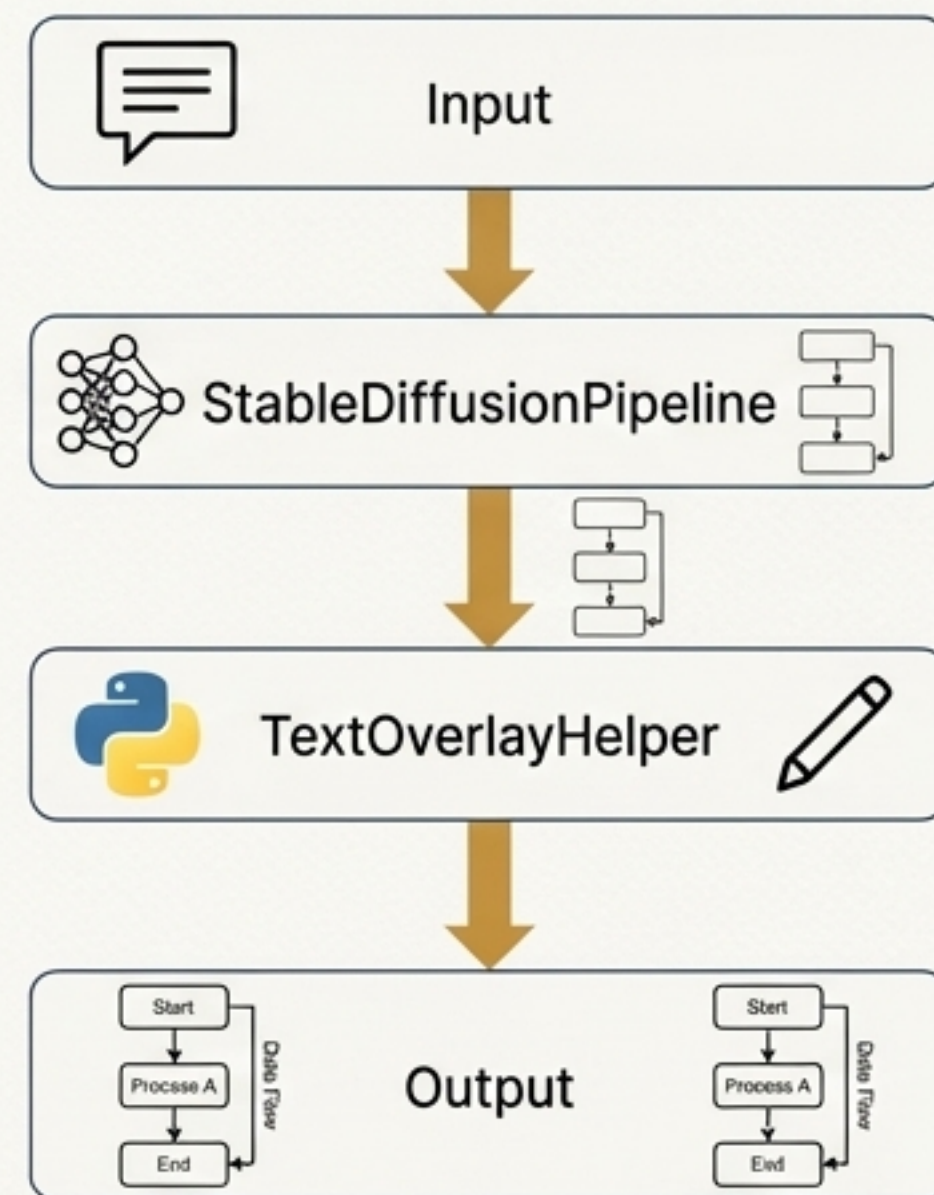
The loss is used to update only the LoRA weights in the `UNet`.

Phase 3: The Two-Step Inference Pipeline

The `generate_and_annotate` function orchestrates the final output. It combines the generative power of the fine-tuned model with the precision of the text rendering helper.

Inference Flow:

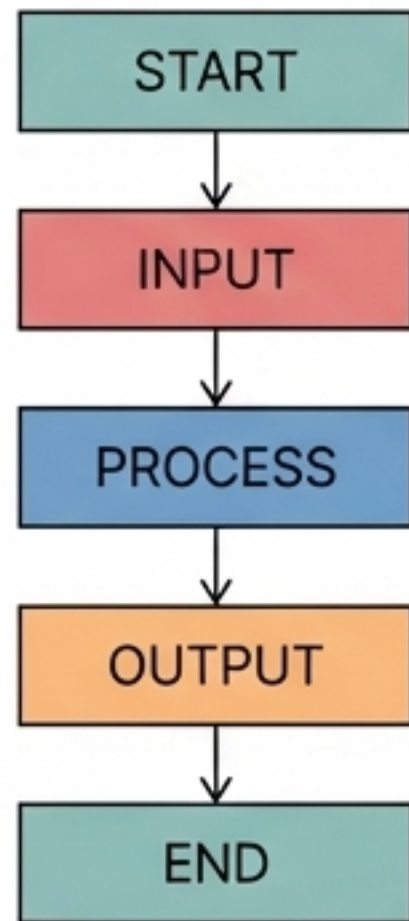
1. **Input:** A text prompt (e.g., 'flowchart showing vertical process flow') and an annotation type (e.g., 'flowchart').
2. **Generate Base Image:** The fine-tuned `StableDiffusionPipeline` generates a text-free visual based on the prompt.
3. **Select Annotation:** The `annotation_type` parameter calls the corresponding method from the `TextOverlayHelper` (e.g., `add_flowchart_labels`).
4. **Apply Text:** The helper method draws the clean, pre-defined text onto the generated image.
5. **Output:** The final, annotated image is returned.



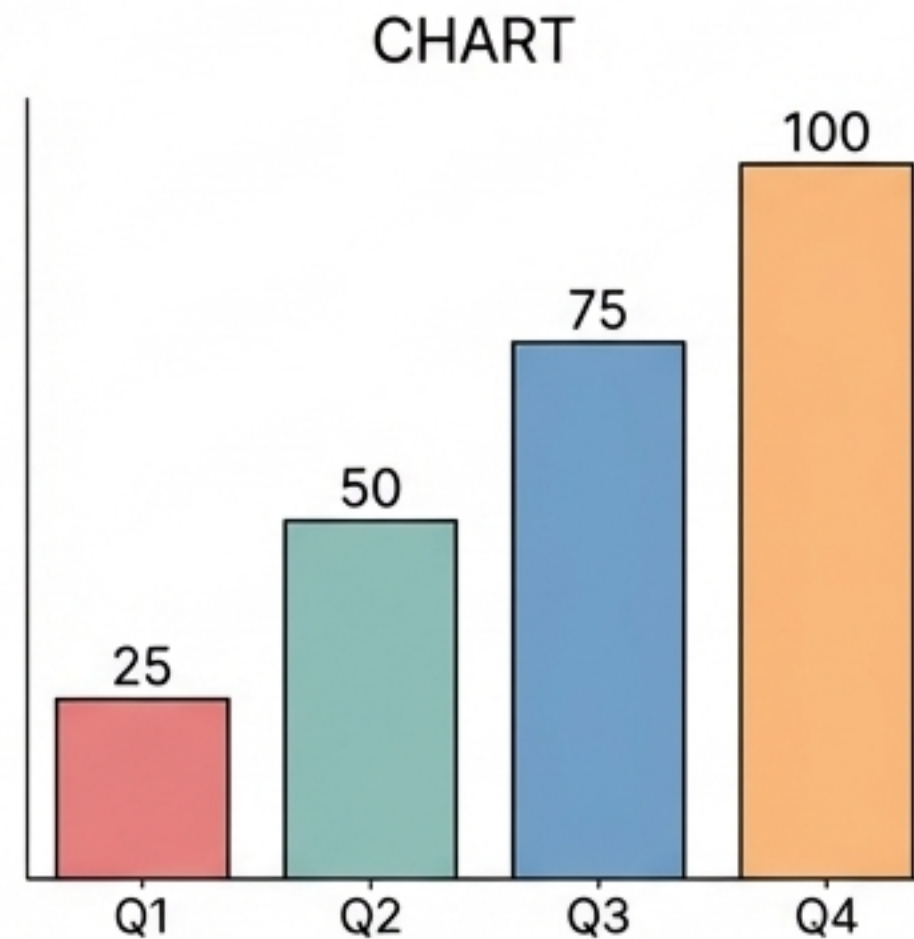
The Results: Consistently Clean and Readable Visualizations

The final pipeline successfully generates a variety of academic visualizations with perfectly rendered text, directly from simple prompts.

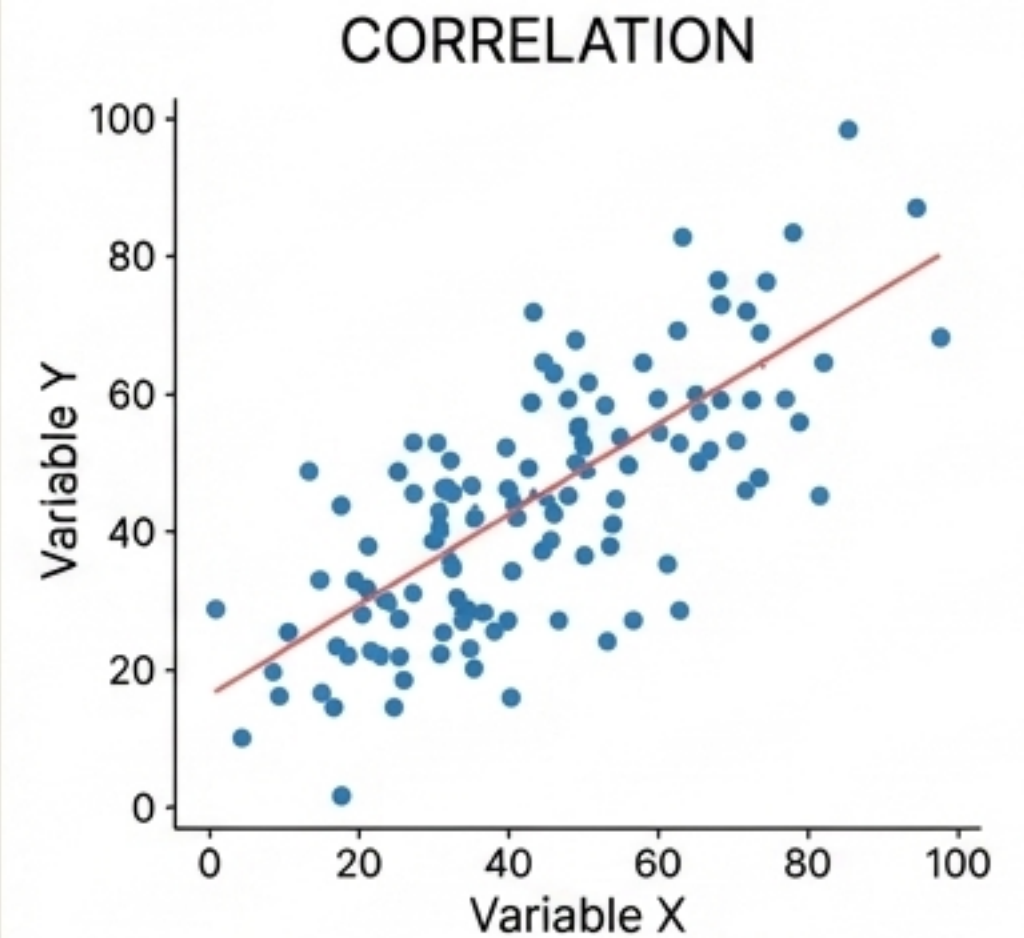
Prompt: "flowchart showing vertical process flow"



Prompt: "bar chart with four colored bars"



Prompt: "scatter plot with blue points"



A Blueprint for Reliable Text in AI-Generated Imagery

By decoupling visual generation from text rendering, this hybrid approach provides a robust and efficient solution to a common failure point in diffusion models.



Reliability

Deterministic text rendering (via PIL) guarantees legibility and positional accuracy, eliminating generative randomness.



Efficiency

LoRA-based fine-tuning allows for rapid specialization of the model with minimal computational cost (<0.4% of parameters trained).



Quality

The synthetic dataset, built on clean Matplotlib visuals and programmatic text, forms a high-quality foundation for training.



Modularity

The separation of the image generator and the text helper makes the system easy to maintain and extend.